

Звіт з лабораторної роботи №4

1. Тема та мета

Тема: Code Review та рефакторинг коду.

Мета: навчитись проводити взаємне рецензування коду, виявляти проблеми якості, документувати їх та виконувати рефакторинг з підтвердженням регресійними тестами.

Виконала: Яцюк Єлизавета (група ПЗПІ-25-1)

Репозиторій: <https://github.com/AnnaYevsovych/CratlogGroup4>

2. Результати Code Review

№	Файл / Рядок	Проблема	Категорія	Рекомендація
1	Portfolio.java, р.8	MAX_WORKS=50 жорстко закодовано, неможливо змінити без перекомпіляції	Magic constant	Передавати maxWorks як параметр конструктора
2	Work.java, р.44	getDetails() дублює toString() — порушує Java-конвенції	Redundant method	Перейменувати на @Override toString()
3	Creator.java, р.57	verify() приймає будь-який 6-значний код як валідний	Logic smell	Додати параметр expectedCode для реального порівняння
4	Portfolio.java, р.75	removeWork повертає boolean але false ніколи не повертається	Inconsistent API	Або повертати false при відсутності, або змінити тип на void
5	CreatorTest.java, р.18	Пароль хардкодом у тесті — ускладнює підтримку	Test smell	Винести в константу: private static final String TEST_PASSWORD

3. Посилання на PR

PR з коментарями Code Review: <https://github.com/AnnaYevsovych/CratlogGroup4/pull/5>

4. Результати рефакторингу

Рефакторинг 1: Додано final до поля workId (Work.java)

БУЛО:

```
private int workId;
```

СТАЛО:

```
private final int workId;
```

ЧОМУ: Ідентифікатор роботи не повинен змінюватись після створення об'єкта. `final` гарантує immutability.

Рефакторинг 2: Замінено `String.matches()` на `Pattern.compile()` (`Creator.java`)

БУЛО:

```
email.trim().matches(EMAIL_REGEX)
```

СТАЛО:

```
EMAIL_PATTERN.matcher(email.trim()).matches() // + додано: private  
static final Pattern EMAIL_PATTERN = Pattern.compile(EMAIL_REGEX);
```

ЧОМУ: `String.matches()` компілює regex при кожному виклику. `Pattern.compile()` робить це один раз — ефективніше.

Рефакторинг 3: Додано валідацію username (`Creator.java`)

БУЛО:

```
this.username = username;
```

СТАЛО:

```
if (username == null || username.isBlank()) {      throw new  
IllegalArgumentException("Username не може бути порожнім"); }  
this.username = username;
```

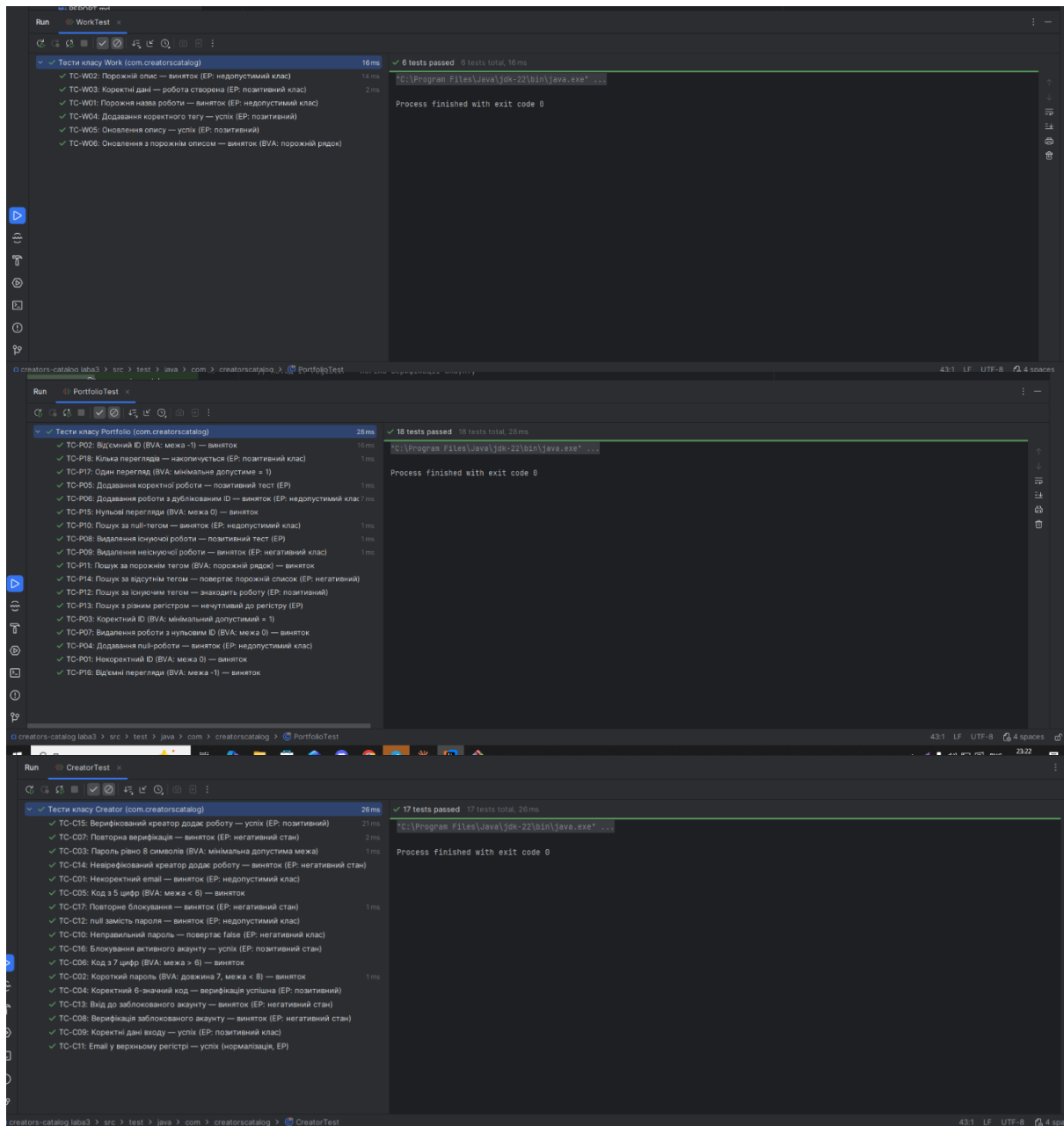
ЧОМУ: Конструктор валідував email і пароль, але не username. Додана перевірка усуває можливий `NullPointerException`.

5. Регресійне тестування

Після рефакторингу запущено всі JUnit-тести:

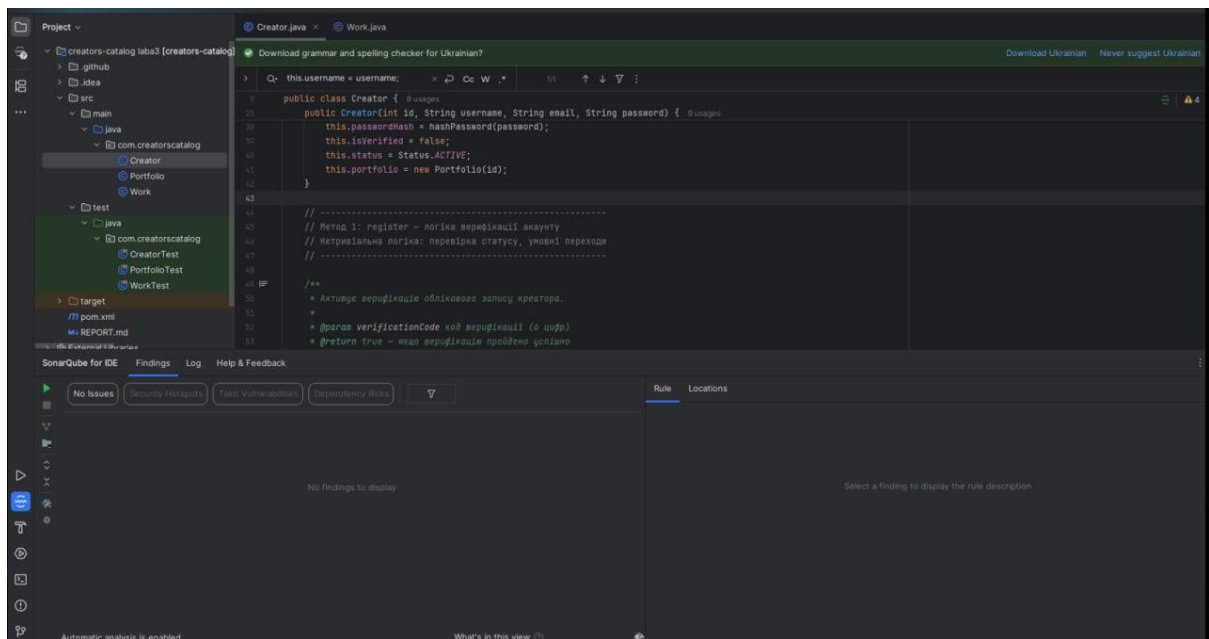
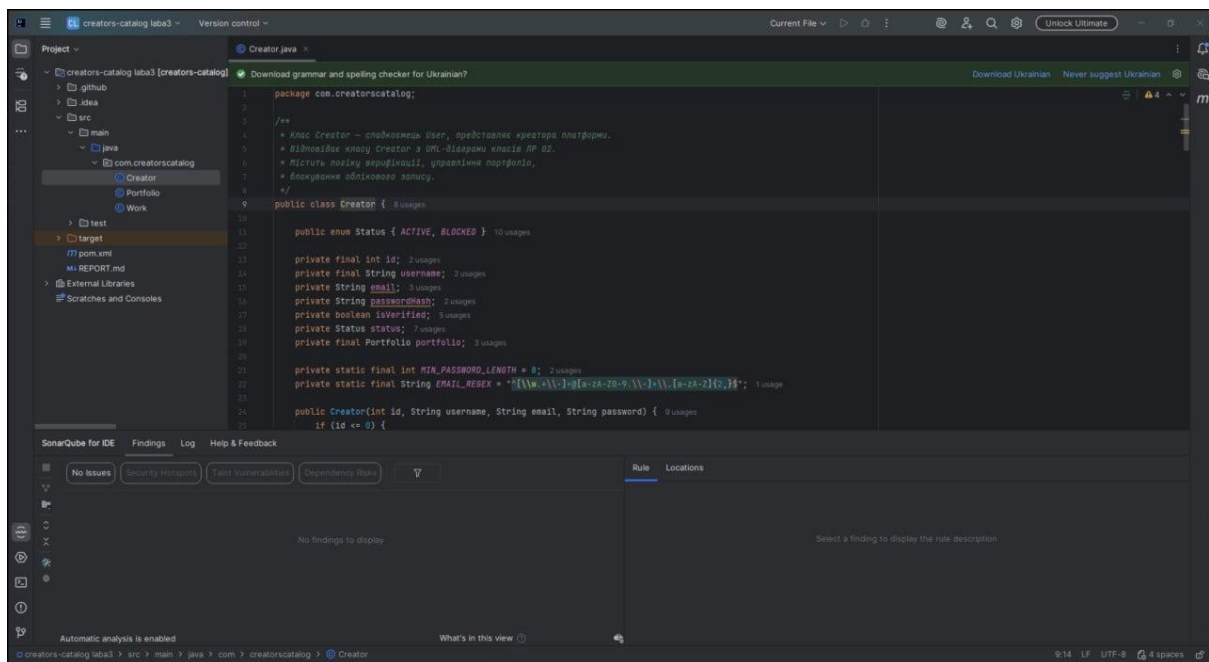
- `CreatorTest`: 17 tests passed
- `PortfolioTest`: 18 tests passed
- `WorkTest`: 6 tests passed

Всього: 41 тест — усі GREEN. Process finished with exit code 0.



6. Порівняння метрик ДО / ПІСЛЯ

Метрика	ДО	ПІСЛЯ
SonarLint issues	0	0
Кількість тестів	41	41
Всі тести green	Так	Так
Валідація username	Відсутня	Додана
Regex компіляція	При кожному виклику	Один раз (Pattern.compile)
Поле workId	Mutable (int)	Immutable (final int)



7. Підсумкова рефлексія

Code Review виявився складнішим ніж я очікувала — потрібно не просто знайти помилку, а пояснити чому це проблема і як її виправити. Найціннішим став досвід аналізу API-консистентності: метод `removeWork()` оголошений як `boolean`, але поводить себе як `void` — це тонка проблема яку легко пропустити. Регресійне тестування підтвердило що рефакторинг не зламав жодного з 41 тестів. Взаємне рецензування допомагає бачити код з іншої перспективи.